

AeroBook Flight Platform

Complete Use Case Diagram & System Specification

Detailed architectural breakdown of Actors, System Boundaries, Microservice Mappings, and Core Business Logic Flows.

PLATFORM ARCHITECTURE AT A GLANCE

Gateway Engine: Spring Cloud Gateway (8083)	Downstream Services: 6 Dedicated Spring Microservices	Database Architecture: Isolated Per-Service Schemas
Security Protocol: JWT Bearer & BCrypt Hashing	Message Broker: RabbitMQ AMQP Broker (5672)	Visual Theme: Black Canvas & White Marker

Executive Summary

AeroBook is a modern, enterprise-grade airline reservation and flight operations platform engineered with a reactive, multi-module microservice topology. To guarantee clear separation of concerns, high scalability, and strict security compliance, the platform distributes domain responsibilities across 7 containerized services: API Gateway (port 8083), Auth Authority (port 8082), User Profile (port 8084), Flight Scheduling & Fleet (port 8087), Booking Engine (port 8088), Dynamic Fare & Pricing (port 8089), and Airport Check-In & Gate Boarding (port 8090).

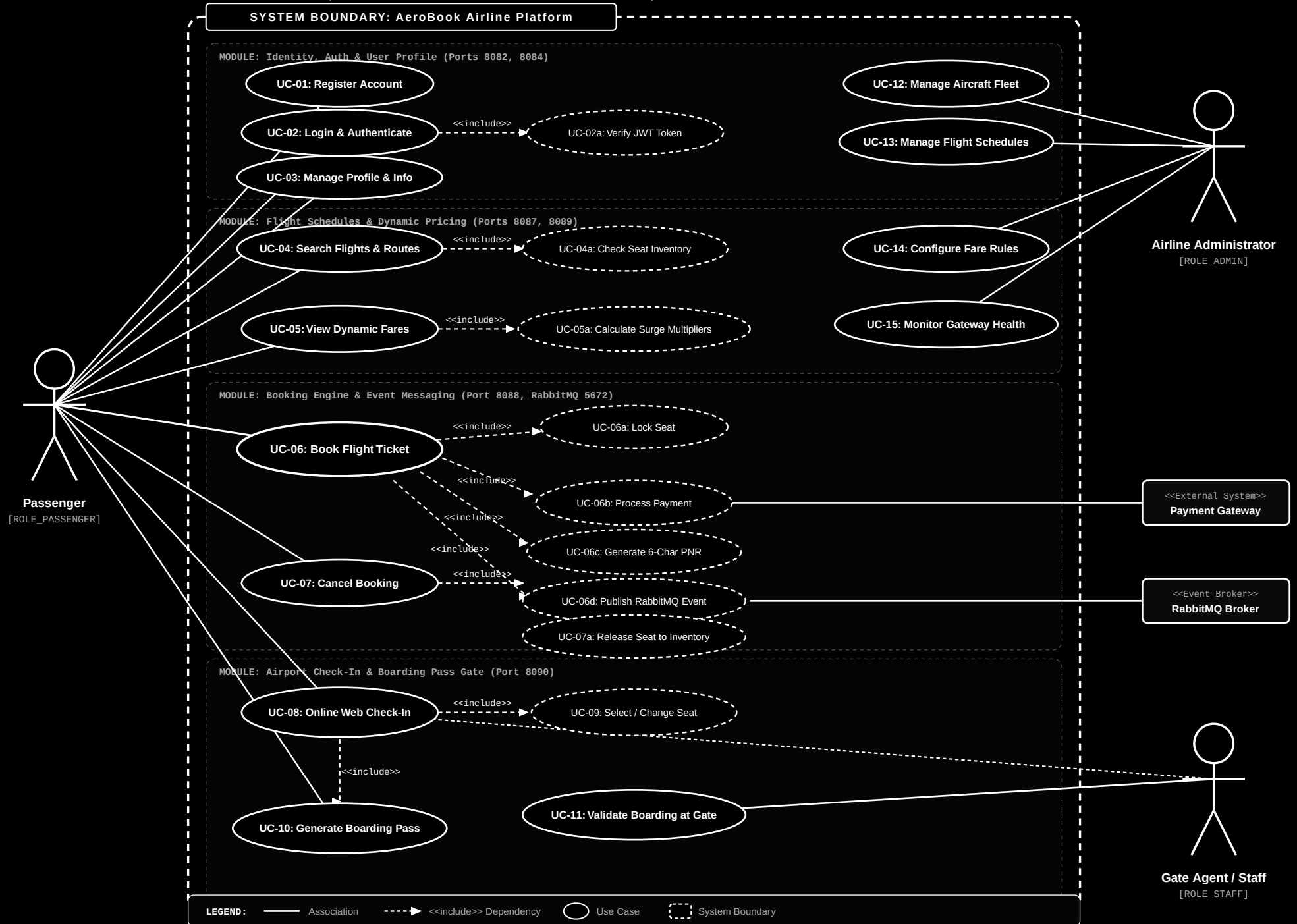
This specification document provides a comprehensive, easy-to-understand reference for the system's Use Case

DOCUMENT CONTENTS & NAVIGATION

Page 2	High-Resolution UML Use Case Diagram (Full Landscape Vector) Complete visual blueprint in dark canvas & white marker theme
Page 3	Section 1: Fundamentals of the Use Case Diagram Plain English definition of actors, boundaries, and use cases
Page 4	Section 2: Detailed Platform Actors & Role Security Passenger, Airline Administrator, Gate Staff, Payment & RabbitMQ
Page 5	Section 3: Core Subsystems & 15 Use Cases Explained Identity, Flight Discovery, Dynamic Pricing & Reservations
Page 6	Section 4: Airport Check-In, Boarding & Admin Operations Web Check-in, QR Boarding Passes, Fleet & Route Scheduling
Page 7	Section 5: End-to-End Walkthroughs & Traceability Matrix Passenger lifecycle, admin operations, and full API endpoint matrix

AEROBOOK PLATFORM - SYSTEM USE CASE DIAGRAM

Enterprise Microservices Architecture • UML 2.5 Specification • Dark Canvas & White Marker Theme



1. Fundamentals: What is this Use Case Diagram?

Core UML 2.5 Concepts Translated into Plain English

A Use Case Diagram is an architectural blueprint that describes WHAT a software system does from the perspective of its users, without getting tangled up in internal programming mechanics. It answers three fundamental questions:

- * Who interacts with the platform? (The Actors)
- * What specific business goals do they accomplish? (The Use Cases)
- * Where does the platform begin and end? (The System Boundary)

The Actors (Stick Figures)

Represent human users or external systems that initiate actions or receive notifications. In our diagram, actors stand outside the boundary because they interact with AeroBook.

The System Boundary (Dashed Box)

The large white dashed rectangle demarcating what is built inside the AeroBook microservice platform versus external third parties like banks or messaging brokers.

Use Cases (White Ellipses)

Each oval represents a discrete, complete business action that produces measurable value for an actor (e.g. "Book Flight Ticket" or "Generate Boarding Pass").

2. Platform Actors: Who Uses AeroBook?

Complete Breakdown of Primary Humans and Secondary Systems

Passenger (Customer / Traveler)

ROLE_PASSENGER & Public Guests

Primary Human Actor (Left Side)

The central customer. Can anonymously browse flight schedules and fares, or authenticate to reserve tickets, select seats, initiate payments, manage bookings, perform 24-hour web check-in, and download digital boarding passes with QR codes.

Airline Administrator

ROLE_ADMIN (High Privilege)

Primary Human Actor (Top Right)

The operational controller of the airline. Responsible for registering aircraft fleet assets (Boeing 737, Airbus A320), configuring cabin seat layouts, creating daily flight route schedules, establishing dynamic pricing formulas, and auditing gateway metrics.

Airport Gate Agent / Staff

ROLE_STAFF (Operational Privilege)

Primary Human Actor (Bottom Right)

Ground operations personnel stationed at airport departure gates. Uses the check-in system to retrieve passenger records by PNR, scan boarding pass QR codes, verify passenger identity, and record gate boarding status before aircraft departure.

External Payment Gateway

External Bank / PCI Provider

Secondary System Actor (Middle Right)

Third-party financial engine (e.g., Stripe, PayPal, Banking API) that securely authorizes card transactions during ticket reservation and issues refunds when a booking is cancelled.

RabbitMQ Message Broker

AMQP Event Messaging (Port 5672)

Secondary System Actor (Middle Right)

Decoupled asynchronous event broker that ingests booking creation events and triggers background notification services (email receipts, SMS alerts, and departure updates) without delaying the user's checkout HTTP response.

3. Subsystems & Use Cases Explained (Part 1)

Identity, Authentication, Flight Discovery & Dynamic Pricing

MODULE 1: Identity, Auth & User Profile (Ports 8082, 8084)

Handles credential security, password hashing, JWT issuance, and passenger profile details.

UC-01: Register Account

auth-service & user-service (POST /api/auth/register)

Collects user email, fullName, and plaintext password. auth-service hashes the password with BCrypt (12 rounds) and user-service stores personal traveler attributes. Assigns default ROLE_PASSENGER.

UC-02: Login & Authenticate

auth-service (POST /api/auth/login)

Verifies email and password. Generates a signed, encrypted JWT Bearer access token valid for 24 hours containing user ID and assigned role authorities.

UC-02a: Verify JWT Token (<<include>>)

api-gateway & auth-service (GET /api/auth/credentials)

Executed automatically on every incoming request passing through the API Gateway. Decodes JWT signature, extracts claims, and blocks tampered or expired requests.

UC-03: Manage Profile & Info

user-service (GET/PUT /api/v1/users/{id})

Allows authenticated travelers to review and update personal details, emergency contacts, passport numbers, and frequent flyer preferences.

MODULE 2: Flight Schedules & Dynamic Pricing (Ports 8087, 8089)

Powers real-time flight inventory searching and automated yield management algorithms.

UC-04: Search Flights & Routes

flight-service (GET /api/flights/search)

Allows passengers to search flights by origin airport, destination airport, and departure date. Returns matched flights, departure/arrival times, aircraft model, and seat availability.

UC-04a: Check Seat Inventory (<<include>>)

flight-service (GET /api/flights/{flightNumber}/seats)

Queries the aircraft cabin seat map to confirm real-time seat counts across Economy, Premium Economy, Business, and First Class cabins.

UC-05: View Dynamic Fares

fare-service (POST /api/fares/calculate)

Calculates the real-time ticket price for a selected flight. Considers base fare, chosen cabin class multiplier, and baggage allowance.

UC-05a: Calculate Surge Multipliers (<<include>>)

fare-service (Internal Pricing Rule Engine)

Automatically applies dynamic pricing algorithms: if seat capacity is over 80% or departure is under 3 days, surge multiplier increases ticket price proportionally.

UML RULE IN PLAIN ENGLISH: What does <<include>> mean?

An <<include>> dependency means the base use case CANNOT complete without executing the included use case. For example, when a passenger executes UC-04 (Search Flights), the system ALWAYS executes UC-04a (Check Seat Inventory) as an essential, non-optional step to present accurate real-time seat counts.

4. Subsystems & Use Cases Explained (Part 2)

The Core Booking Engine & Airport Check-In Services

MODULE 3: Booking Engine & Event Messaging (Port 8088)

The critical financial and transactional hub of the AeroBook platform.

UC-06: Book Flight Ticket (Master Use Case)

booking-service (POST /api/bookings)

The master transaction. Orchestrates the reservation workflow by coordinating seat locking, payment execution, PNR generation, and messaging event publication.

UC-06a: Lock Seat (<<include>>)

flight-service (POST /api/flights/{flightNumber}/seats/lock)

Temporarily reserves the requested seat for 10 minutes in flight-service. Prevents double-booking race conditions while the passenger enters payment details.

UC-06b: Process Payment (<<include>>)

booking-service & External Payment Gateway API

Dispatches credit card transaction to the external Payment Gateway. If payment succeeds, booking transitions to CONFIRMED. If payment fails, the seat lock is aborted.

UC-06c: Generate 6-Char PNR (<<include>>)

booking-service (PnrGenerator Utility)

Generates an immutable, collision-resistant 6-character alphanumeric Passenger Name Record (e.g. AB7K9Q) used as the master booking reference across all services.

UC-06d: Publish RabbitMQ Event (<<include>>)

booking-service (RabbitTemplate AMQP Exchange: booking.exchange)

Publishes BookingCreatedEvent payload onto RabbitMQ queue booking.events. Allows asynchronous workers to send email confirmations and SMS tickets without latency.

UC-07: Cancel Booking & Refund

booking-service (DELETE /api/bookings/{id})

Cancels confirmed reservation, calculates cancellation refund amount, and initiates an asynchronous seat release back to the available inventory pool.

UC-07a: Release Seat to Inventory (<<include>>)

flight-service (DELETE /api/flights/{flightNumber}/seats/lock)

Notifies flight-service to decrement reserved seat counter and make the seat instantly bookable by other passengers.

MODULE 4: Airport Check-In & Boarding Pass Gate (Port 8090)

Handles the final pre-flight phase: passenger check-in, seat assignment, and gate boarding.

UC-08: Online Web Check-In

check-in-service (POST /api/check-ins)

Accessible 24 hours prior to departure. Passenger enters PNR and surname to check in and initialize the digital boarding pass.

UC-09: Select / Change Seat (<<include>>)

check-in-service (POST /api/check-ins/select-seat)

Interactive seat assignment: passenger picks specific seat (Window 12A, Aisle 12B, Extra Legroom 1A) and updates check-in record.

UC-10: Generate Boarding Pass

check-in-service (GET /api/check-ins/{id}/boarding-pass)

Renders digital boarding pass with passenger name, PNR, flight number, seat number, gate, boarding time, and encoded security QR code.

UC-11: Validate Boarding at Gate

check-in-service (PUT /api/check-ins/{id}/board)

Gate agent scans the QR code at the gate. System validates security hash and transitions passenger status to BOARDED.

5. Administration Operations & Real-World Flows

Fleet Management, Pricing Rules, and Step-by-Step Journeys

ADMINISTRATOR USE CASES (ROLE_ADMIN Only)

UC-12: Manage Aircraft Fleet Registers new aircraft (Boeing 737, Airbus A320), sets seat counts per cabin, and marks maintenance status.	<code>flight-service (POST/GET/PUT /api/admin/aircrafts)</code>
UC-13: Manage Flight Schedules Creates new routes, flight numbers, departure/arrival airport codes, gate numbers, and departure times.	<code>flight-service (POST/GET/PUT /api/admin/flights)</code>
UC-14: Configure Fare Rules Sets base route prices, holiday peak multipliers, cabin class upgrade coefficients, and extra baggage fees.	<code>fare-service (POST/PUT /api/fares)</code>
UC-15: Monitor Gateway Health Audits reactive gateway routing health, microservice directory, and active RBAC security rules.	<code>api-gateway (GET /api/gateway/health & /api/gateway/services)</code>

End-to-End Real-World Scenario Walkthroughs

How Use Cases Interact in Production

JOURNEY 1: The Traveler Lifecycle (Booking to Boarding Gate)

- Flight Discovery: Passenger executes UC-04 (Search Flights) -> System checks seat inventory UC-04a.
- Dynamic Price Quote: Passenger views fare UC-05 -> System applies demand surge multipliers UC-05a.
- Reservation Master Transaction: Passenger clicks Book UC-06 ->
 - flight-service locks requested seat UC-06a for 10 minutes.
 - Payment Gateway authorizes transaction UC-06b.
 - booking-service generates unique 6-character PNR (e.g. AB7K9Q) UC-06c.
 - booking-service publishes event to RabbitMQ UC-06d (triggers email receipt).
- Check-In & Boarding (24h Pre-flight): Passenger accesses UC-08 (Web Check-In) -> selects seat UC-09 -> downloads digital QR boarding pass UC-10 -> Gate staff scans QR code at departure gate UC-11 to mark passenger as BOARDED.

JOURNEY 2: The Airline Operational Lifecycle (Fleet & Yield Management)

- Fleet Commissioning: Admin executes UC-12 to register a new Boeing 737 with 180 seats.
- Flight Route Scheduling: Admin executes UC-13 to schedule flight AB-204 from JFK to LHR.
- Pricing Strategy Setup: Admin executes UC-14 in fare-service to set base fare at \$450 with 1.8x Business multiplier.
- Platform Health Check: Admin audits UC-15 via API Gateway to ensure all 6 microservices are healthy and registered.

6. Microservice Traceability Matrix & Evaluation FAQ

Academic, Defense & Architecture Review Quick Reference

UC ID	Use Case Name	Primary Actor	Microservice & Port	Target Endpoint
UC-01	Register Account	Passenger	auth-service (8082)	POST /api/auth/register
UC-02	Login & Authenticate	All Actors	auth-service (8082)	POST /api/auth/login
UC-03	Manage Profile	Passenger	user-service (8084)	GET /api/v1/users/{id}
UC-04	Search Flights	Passenger	flight-service (8087)	GET /api/flights/search
UC-05	View Dynamic Fares	Passenger	fare-service (8089)	POST /api/fares/calculate
UC-06	Book Flight Ticket	Passenger	booking-service (8088)	POST /api/bookings
UC-07	Cancel Booking	Passenger	booking-service (8088)	DELETE /api/bookings/{id}
UC-08	Online Web Check-In	Passenger/Staff	check-in-service (8090)	POST /api/check-ins
UC-09	Select Seat	Passenger	check-in-service (8090)	POST /check-ins/select-seat
UC-10	Generate Boarding Pass	Passenger	check-in-service (8090)	GET /check-ins/{id}/pass
UC-11	Validate Gate Boarding	Gate Staff	check-in-service (8090)	PUT /api/check-ins/{id}/board
UC-12	Manage Aircrafts	Administrator	flight-service (8087)	POST /api/admin/aircrafts
UC-13	Manage Flights	Administrator	flight-service (8087)	POST /api/admin/flights
UC-14	Configure Fares	Administrator	fare-service (8089)	POST /api/fares
UC-15	Gateway Diagnostics	Administrator	api-gateway (8083)	GET /api/gateway/health

Frequently Asked Architectural & Viva Questions

Key Defense Points for Evaluators

Q1: Why are Payment Gateway and RabbitMQ modeled outside the System Boundary?

Because they are external secondary systems. AeroBook interacts with them over network protocols (REST HTTP and AMQP), but their internal lifecycle is managed outside our core microservice boundary.

Q2: Why is <<include>> used instead of <<extend>> for UC-06a (Lock Seat) and UC-06c (PNR)?

In UML, <<include>> denotes a mandatory step that ALWAYS executes as part of the parent use case. Booking cannot legally exist without a locked seat and a valid PNR. If either fails, the booking cannot complete.

Q3: How does the Use Case diagram map to our Spring Cloud Microservices?

Each subsystem box inside the boundary corresponds 1-to-1 with a Spring Boot microservice running on an isolated port with its own database schema. The API Gateway unifies all these use cases under port 8083.

